

Vtable and Vptr

Yes — this is the **internal mechanism behind runtime polymorphism** in C++. For interviews, you don't need compiler-level assembly details, but you should understand **vp**tr → **vt**able → **virtual function** clearly.

1. First, the problem vtable solves

Suppose:

```
class Animal {
public:
    virtual void speak() {
        cout << "Animal\n";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Dog\n";
    }
};
```

And:

```
Animal* p = new Dog();

p->speak();
```

At compile time, **p** is:

```
Animal*
```

but the actual object is:

```
Dog
```

C++ needs a mechanism to say:

"The pointer is an **Animal***, but the object is actually a **Dog**, so call **Dog::speak()**."

That's where **vt**able + **vp**tr come in.

2. What is a vtable?

A **virtual table (vtable)** is essentially a compiler-generated table containing pointers to the virtual functions for a class.

Conceptually:

```
Animal vtable
┌───────────────────┐
│ Animal::speak()    │
└───────────────────┘
```

For **Dog**:

```
Dog vtable
┌───────────────────┐
│ Dog::speak()       │
└───────────────────┘
```

Important:

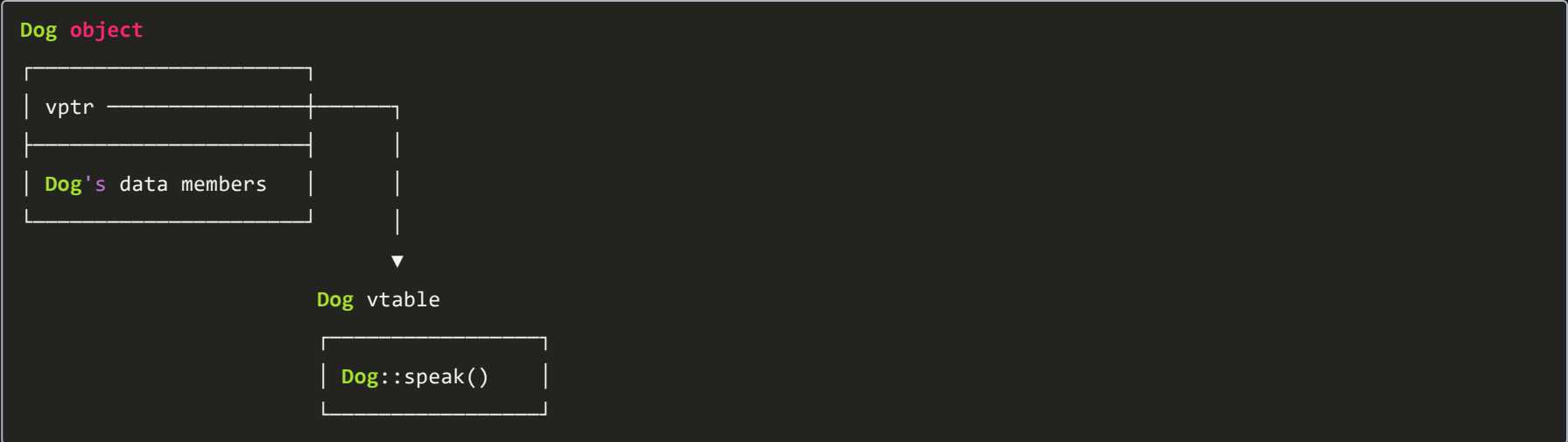
A class typically has a vtable; individual objects don't each have their own separate vtable.

3. What is vp

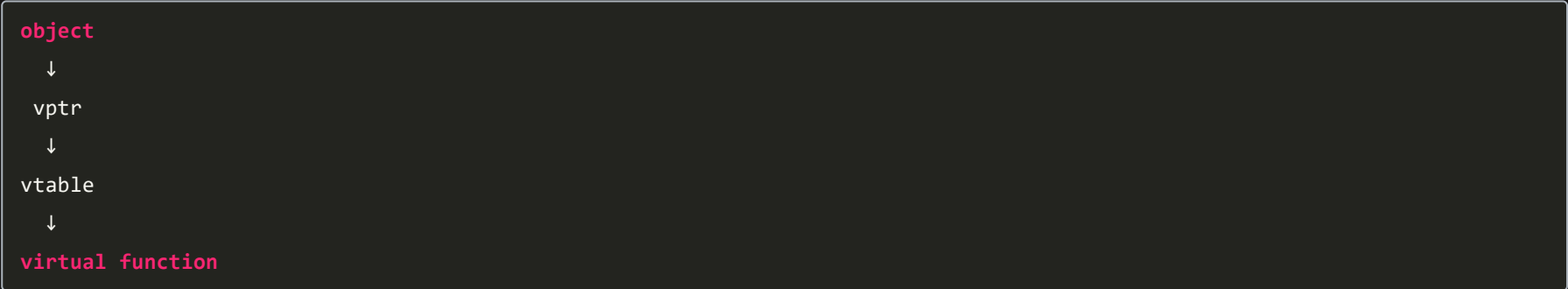
Each polymorphic object typically contains a hidden pointer called a **vp**tr.

The vp

Conceptually:



So:



That's the entire mental model.

4. Let's trace the actual call

Consider:

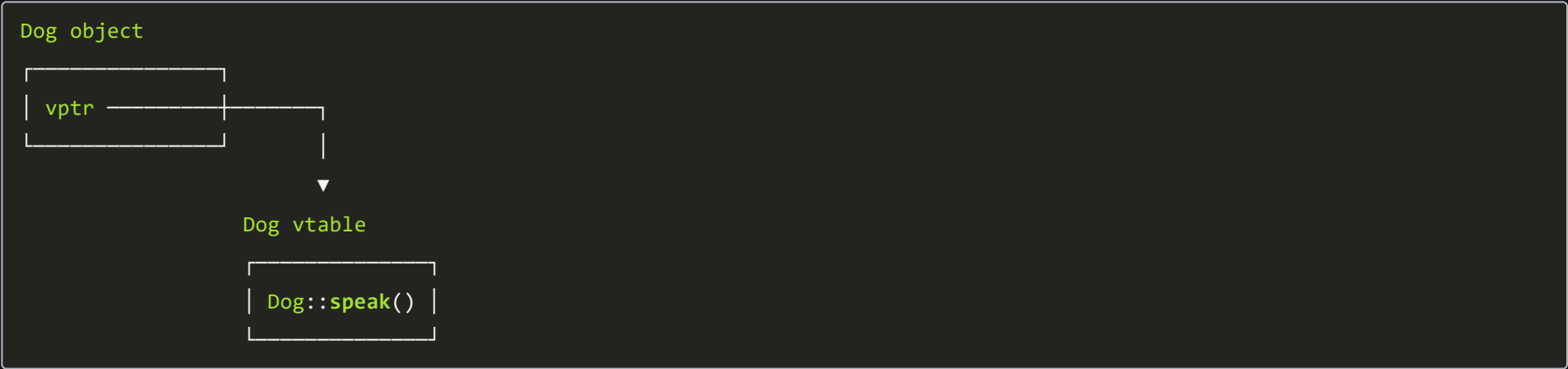
```
Animal* p = new Dog();

p->speak();
```

Step 1 — Create Dog

When the **Dog** object is created, its vptr is set to point to the **Dog** vtable.

Conceptually:



Step 2 — Store it in base pointer

```
Animal* p = new Dog();
```

p points to the same object.



The pointer's **static type** is **Animal***.

But the object's **dynamic type** is **Dog**.

Step 3 — Call virtual function

```
p->speak();
```

Because **speak()** is virtual, the runtime effectively performs something conceptually like:



Therefore:

Dog

gets printed.

5. What if the object is Animal?

```
Animal* p = new Animal();

p->speak();
```

Now:



So:

Animal

gets printed.

6. Why doesn't every function use a vtable?

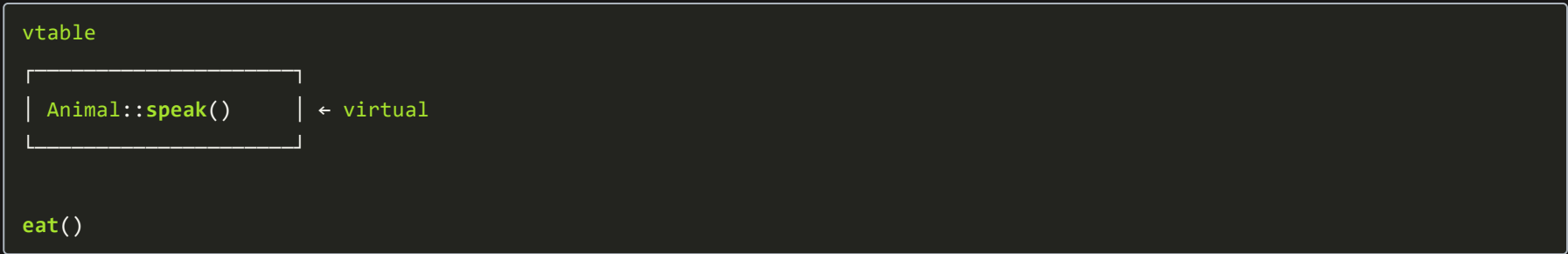
Because only **virtual functions** require dynamic dispatch.

For:

```
class Animal {
public:
    void eat() {
        cout << "eat";
    }

    virtual void speak() {
        cout << "speak";
    }
};
```

Conceptually:



doesn't need to be in the vtable because it isn't virtual.

The compiler can resolve:

p->eat();

statically.

7. What happens with inheritance?

Consider:

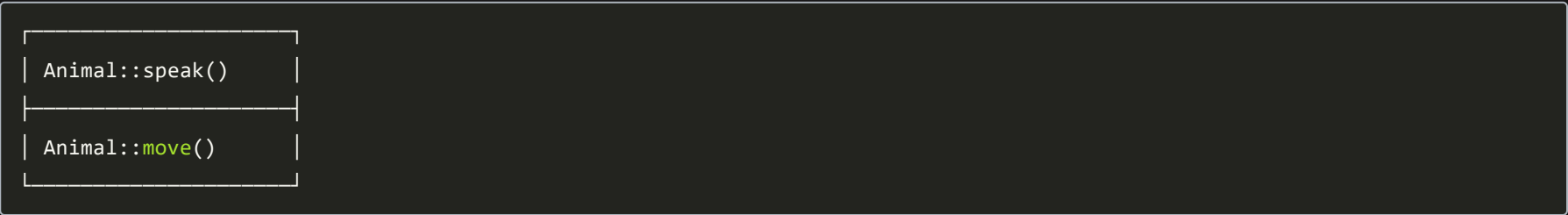
```
class Animal {
public:
    virtual void speak() {
        cout << "Animal";
    }

    virtual void move() {
        cout << "Animal moves";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Dog";
    }
};
```

Conceptually:

Animal vtable



Dog vtable

Since Dog overrides only `speak()`:



So:

```
Animal* p = new Dog();

p->speak(); // Dog
p->move();  // Animal moves
```

The vtable contains the appropriate implementation for each virtual function.

8. What does a vtable actually contain?

For interview purposes, say:

"It contains function pointers or entries that allow the implementation to dispatch virtual calls."

Don't overclaim that it contains **only** function pointers.

Real compiler implementations may put additional implementation-specific information in or around the vtable, such as RTTI-related information, offsets, etc.

The C++ standard itself does **not mandate** vtables.

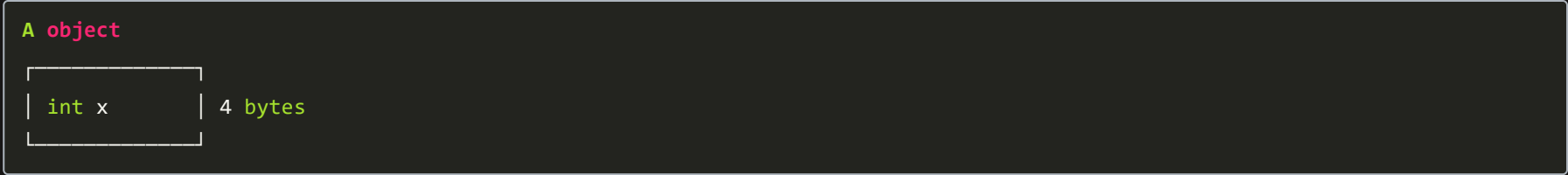
This is an implementation technique used by common compilers.

9. Why does vptr add memory overhead?

Suppose:

```
class A {
public:
    int x;
};
```

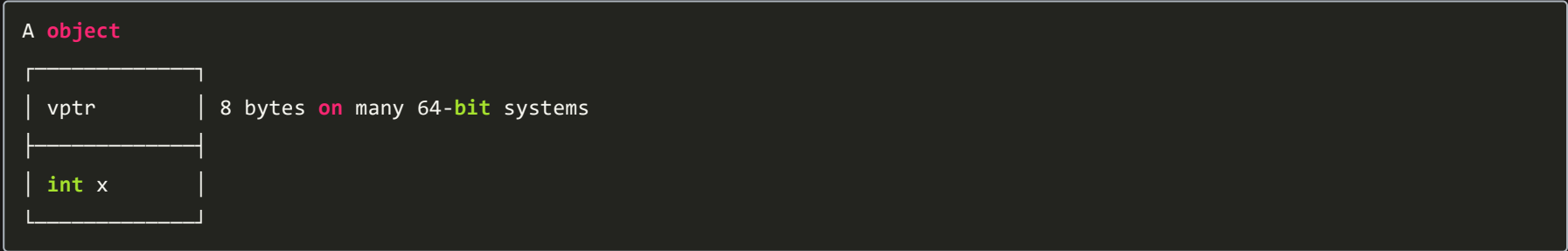
An object might conceptually contain:



Now:

```
class A {
public:
    virtual void foo();
    int x;
};
```

A typical implementation adds a hidden vptr:



So a polymorphic object typically has extra memory overhead for the vptr.

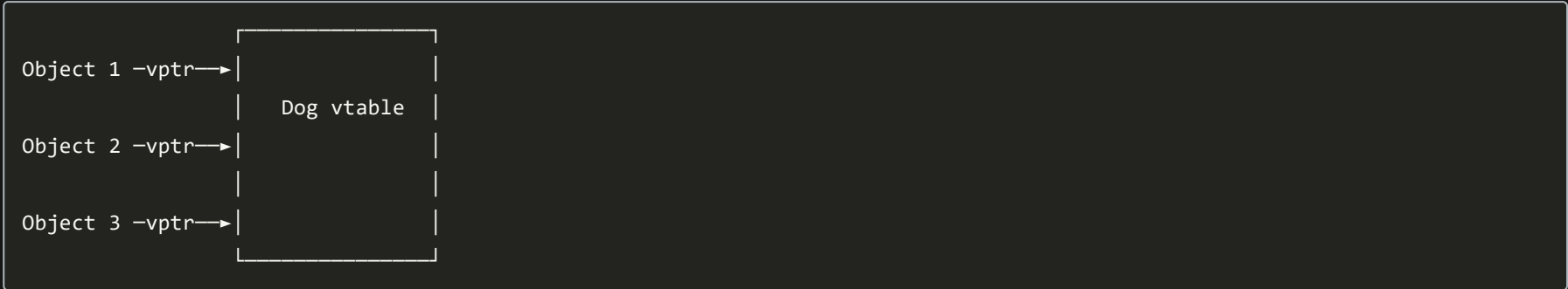
The exact object size depends on ABI, alignment, compiler, architecture, inheritance, etc.

10. Does every object have its own vtable?

No.

This is a very common interview trap.

Usually:



Multiple Dog objects can share the same Dog vtable.

Each object typically has its **own vptr**, but the vtable can be shared.

11. What happens when constructing objects?

This is a slightly deeper interview question.

Consider:

```
class Base {
public:
    Base() {
        foo();
    }

    virtual void foo() {
        cout << "Base";
    }
};

class Derived : public Base {
public:
    void foo() override {
        cout << "Derived";
    }
};
```

Now:

```
Derived d;
```

You might expect:

Derived

But the result is:

Base

Why?

During construction of the **Base** portion, the object is treated as a **Base** object for virtual dispatch purposes.

Conceptually:

```
Base constructor
  ↓
Base vptr
  ↓
Base::foo()
```

Then the Derived constructor runs and the vptr is adjusted to the Derived vtable.

So:

```
Construction:

Base constructor
  ↓
Base vtable
  ↓
Derived constructor
  ↓
Derived vtable
```

This is also why calling virtual functions from constructors/destructors doesn't give you normal derived-class dispatch.

12. What happens during destruction?

Similarly:

```
class Base {
public:
    virtual ~Base() {}
};

class Derived : public Base {
public:
    ~Derived() {}
};
```

When:

```
Base* p = new Derived();
delete p;
```

the virtual destructor ensures:

```
Derived destructor
  ↓
Base destructor
```

is called appropriately.

That's why:

If a class is intended to be used polymorphically, its destructor should generally be virtual.

13. The actual dispatch cost

When you write:

```
p->speak();
```

a simplified conceptual model is:


```
p
↓
read vptr
↓
read function pointer from vtable
↓
indirect function call
```

So compared to a normal direct call:

```
Normal:
compile-time known address
      ↓
      function()
```

Virtual:

```
object
  ↓
  vptr
  ↓
  vtable
  ↓
function pointer
  ↓
function()
```

That's why virtual dispatch can have a small runtime cost.

14. Why can't **static** functions be virtual?

Consider:

```
class Base {
public:
    static void foo();
};
```

Static functions don't operate on a particular object.

There is no:

```
this
```

pointer.

But virtual dispatch depends on the object's:

```
object → vptr → vtable
```

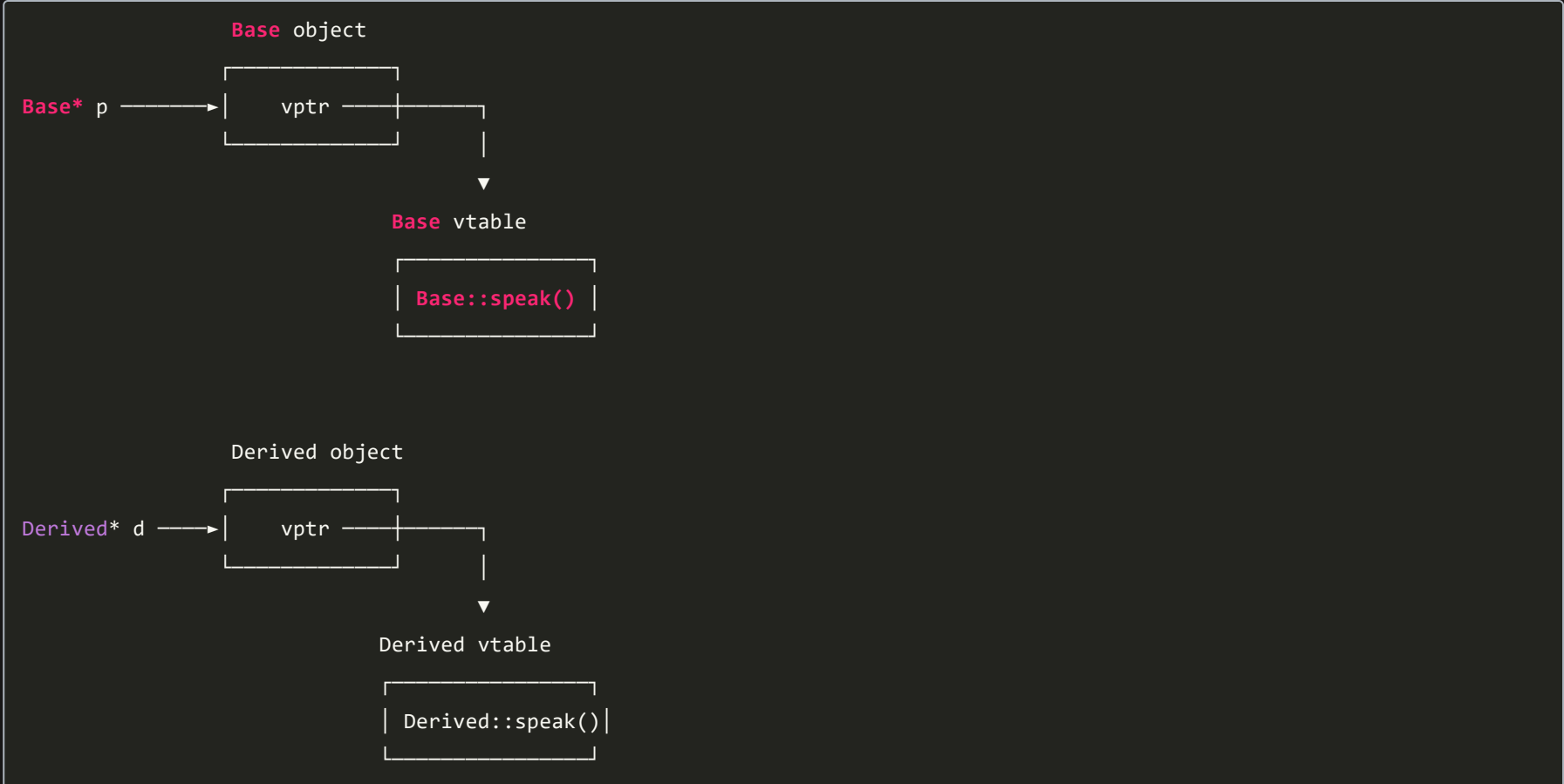
Therefore:

```
static virtual void foo(); // ✕
```

is invalid.

15. The most important interview diagram

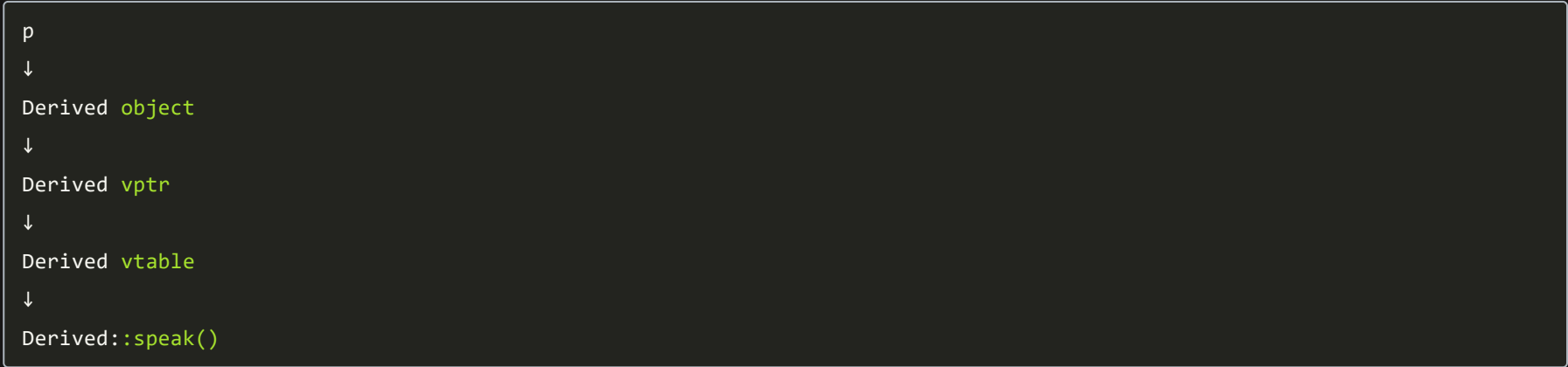
Memorize this:



Then:

```
Base* p = new Derived();
p->speak();
```

means conceptually:



Interview questions you should now prepare

Basic

1. What is a vtable?
2. What is a vptr?
3. Why is a vptr required?
4. Does C++ standard guarantee vtable/vptr?
5. Does every object have its own vtable?
6. Does every class have a vtable?
7. What is the memory overhead of virtual functions?

Deeper

1. How does `Base* p = new Derived(); p->foo();` find `Derived::foo()`?
2. What happens to vptr during construction?
3. Why don't virtual calls from constructors behave polymorphically?
4. Why should a polymorphic base class usually have a virtual destructor?
5. Can you explain the performance cost of virtual dispatch?
6. Can the compiler optimize/devirtualize a virtual call?

Classic output question

Be able to explain this without hesitation:


```

class Base {
public:
    virtual void show() {
        cout << "Base";
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived";
    }
};

int main() {
    Base* p = new Derived();
    p->show();
}

```

Answer: **Derived**

Because:

```

Base*
↓
actual object = Derived
↓
vptr
↓
Derived vtable
↓
Derived::show()

```

The one sentence to remember

A vptr is typically a hidden pointer inside a polymorphic object that points to the class's vtable, and the vtable contains entries used to dynamically dispatch virtual function calls.

That's enough depth for most fresher C++ OOP interviews.